PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL)

Starter Report: 10 Case Studies

Student Name	Geetha Vaishali
Programme	First-Year B.Tech – Computer Science and Engineering
Institution	Dr. RVR & NRI Institute of Technology
Roll Number / Section	12000098
Course	Python Programming

Important note: The college circular says students must select 10 difficult-level case studies from the faculty's prescribed list of 100. That list has not been provided in this chat, so the following are provisional practice examples, not verified selections from the official list. Match these topics to your official list before submission.

1. Introduction

This report demonstrates Python problem-solving through ten practice case studies. Each case study contains a problem statement, objective, assumptions, algorithm, text flowchart, pseudocode, source code, sample test cases and conclusion. The examples use beginner-level Python constructs such as input/output, conditions, loops, lists and functions.

2. Objectives

• Understand a problem and identify inputs and outputs.

• Design an algorithm before writing code.

• Implement and test Python programs.

• Explain the program logic in simple language.

• Organize documentation for an academic repository.

3. How to run the programs

Save each program as a separate .py file and run it with Python 3. For example: `python case01_prime_number.py`. Record your own output screenshots after running each file. The outputs below are expected examples; they are not screenshots.

Case Study 1: Prime Number Checker

Problem statement: Determine whether a positive integer greater than 1 is prime.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Input is an integer. Numbers below 2 are not prime.

Algorithm: Read n; test divisors from 2 while d*d <= n; if any divides n, composite; otherwise prime.

Text flowchart: Start → Read n → Check n < 2? → Test divisors → Print result → End.

Pseudocode: Read n; if n < 2 print not prime; otherwise test divisors up to square root and print result.

Python source code:

```
n = int(input("Enter an integer: "))
if n < 2:
    print(n, "is not prime")
else:
    prime = True
    d = 2
    while d * d <= n:
        if n % d == 0:
            prime = False
            break
        d += 1
    if prime:
        print(n, "is prime")
    else:
        print(n, "is not prime")
```

Test cases / expected output:

Input	Expected result
7	7 is prime
12	12 is not prime
1	1 is not prime

Conclusion: The program checks possible divisors efficiently and classifies the input.

Case Study 2: Armstrong Number Checker

Problem statement: Check whether a non-negative integer equals the sum of its digits each raised to the number of digits.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Input is a non-negative integer.

Algorithm: Count digits; add each digit raised to digit count; compare sum with original number.

Text flowchart: Start → Read n → Count digits → Sum digit powers → Compare → Print → End.

Pseudocode: Store original; find digit count; calculate powered digit sum; compare with original.

Python source code:

```
n = int(input("Enter a non-negative integer: "))
if n < 0:
    print("Please enter a non-negative integer")
else:
    original = n
    digits = len(str(n))
    total = 0
    temp = n
    while temp > 0:
        digit = temp % 10
        total += digit ** digits
        temp //= 10
    if total == original:
        print(original, "is an Armstrong number")
    else:
        print(original, "is not an Armstrong number")
```

Test cases / expected output:

Input	Expected result
153	153 is an Armstrong number
123	123 is not an Armstrong number
0	0 is an Armstrong number

Conclusion: The program uses digit extraction and exponentiation to test the property.

Case Study 3: Palindrome Number

Problem statement: Check whether a non-negative integer reads the same forwards and backwards.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Input is a non-negative integer.

Algorithm: Reverse the digits and compare the reversed value with the original.

Text flowchart: Start → Read n → Reverse digits → Compare → Print → End.

Pseudocode: Keep original; build reverse using remainder and integer division; compare.

Python source code:

```
n = int(input("Enter a non-negative integer: "))
if n < 0:
    print("Please enter a non-negative integer")
else:
    original = n
    temp = n
    reverse = 0
    while temp > 0:
        digit = temp % 10
        reverse = reverse * 10 + digit
        temp //= 10
    if original == reverse:
        print(original, "is a palindrome")
    else:
        print(original, "is not a palindrome")
```

Test cases / expected output:

Input	Expected result
1221	1221 is a palindrome
123	123 is not a palindrome
0	0 is a palindrome

Conclusion: The program reverses the digits without converting the number to a string.

Case Study 4: GCD and LCM

Problem statement: Find the greatest common divisor and least common multiple of two integers.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Inputs are integers and not both zero.

Algorithm: Use Euclid's algorithm for GCD; compute LCM from absolute product divided by GCD.

Text flowchart: Start → Read a,b → Apply Euclid loop → Calculate LCM → Print → End.

Pseudocode: While b is not zero, replace (a,b) with (b,a mod b); use GCD to calculate LCM.

Python source code:

```
a = int(input("Enter first integer: "))
b = int(input("Enter second integer: "))
x, y = abs(a), abs(b)
while y != 0:
    x, y = y, x % y
gcd = x
if gcd == 0:
    print("GCD and LCM are undefined for both inputs zero")
else:
    lcm = abs(a * b) // gcd
    print("GCD =", gcd)
    print("LCM =", lcm)
```

Test cases / expected output:

Input	Expected result
12, 18	GCD = 6; LCM = 36
7, 5	GCD = 1; LCM = 35

Conclusion: Euclid's algorithm finds the GCD efficiently, and the GCD is used to find the LCM.

Case Study 5: Second Largest Distinct Number

Problem statement: Find the second-largest distinct value in a list of integers.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: At least two distinct values must exist.

Algorithm: Remove duplicates, sort values and select the second-last item.

Text flowchart: Start → Read list → Remove duplicates → Check count → Sort → Print second largest → End.

Pseudocode: Convert list to a set; if fewer than two unique values report it; otherwise sort and print the second-last value.

Python source code:

```
numbers = list(map(int, input("Enter integers separated by spaces: ").split()))
unique = sorted(set(numbers))
if len(unique) < 2:
    print("There is no second-largest distinct number")
else:
    print("Second-largest distinct number =", unique[-2])
```

Test cases / expected output:

Input	Expected result
4 1 9 9 6	Second-largest distinct number = 6
5 5 5	There is no second-largest distinct number

Conclusion: The program handles repeated values by considering only distinct numbers.

Case Study 6: Student Grade Calculator

Problem statement: Calculate the average of marks and assign a grade.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Enter marks for five subjects, each from 0 to 100.

Algorithm: Validate marks; compute average; assign grade using stated thresholds.

Text flowchart: Start → Read marks → Validate → Average → Grade decision → Print → End.

Pseudocode: Read five marks; reject invalid values; calculate average and use if/elif conditions.

Python source code:

```
marks = list(map(float, input("Enter five marks separated by spaces: ").split()))
if len(marks) != 5 or any(m < 0 or m > 100 for m in marks):
    print("Enter exactly five marks from 0 to 100")
else:
    average = sum(marks) / len(marks)
    if average >= 90:
        grade = "A"
    elif average >= 80:
        grade = "B"
    elif average >= 70:
        grade = "C"
    elif average >= 60:
        grade = "D"
    else:
        grade = "E"
    print("Average =", round(average, 2))
    print("Grade =", grade)
```

Test cases / expected output:

Input	Expected result
80 85 90 75 70	Average = 80.0; Grade = B
50 55 60 45 40	Average = 50.0; Grade = E

Conclusion: The program combines list processing, validation and conditional statements. Grade boundaries are illustrative.

Case Study 7: Word Frequency Counter

Problem statement: Count how often each word appears in a sentence.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Words are separated by whitespace; punctuation is treated as part of a word in this basic version.

Algorithm: Convert to lowercase and update a dictionary for each word.

Text flowchart: Start → Read sentence → Split words → Update dictionary → Display counts → End.

Pseudocode: Lowercase sentence; split into words; for each word increment dictionary count.

Python source code:

```
sentence = input("Enter a sentence: ").lower()
counts = {}
for word in sentence.split():
    counts[word] = counts.get(word, 0) + 1
for word, count in counts.items():
    print(word, ":", count)
```

Test cases / expected output:

Input	Expected result
Python is fun and Python is useful	python : 2; is : 2; fun : 1; and : 1; useful : 1

Conclusion: A dictionary stores each word as a key and its frequency as the value.

Case Study 8: Inventory Stock Alert

Problem statement: Read item names and quantities, then report items at or below a reorder threshold.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Quantity and threshold are non-negative integers.

Algorithm: Compare each item's quantity with the threshold and print low-stock items.

Text flowchart: Start → Read threshold → Read items → Compare quantities → Report low stock → End.

Pseudocode: Create item list; for each item, compare quantity with threshold; display alert if low.

Python source code:

```
threshold = int(input("Enter reorder threshold: "))
count = int(input("How many items? "))
items = []
for i in range(count):
    name = input("Item name: ")
    quantity = int(input("Quantity: "))
    items.append((name, quantity))
print("Items needing reorder:")
found = False
for name, quantity in items:
    if quantity <= threshold:
        print(name, "-", quantity)
        found = True
if not found:
    print("No items need reorder")
```

Test cases / expected output:

Input	Expected result
Threshold 5; rice 3; soap 8	rice - 3

Conclusion: The program demonstrates lists of tuples and conditional filtering for a practical stock-management task.

Case Study 9: Electricity Bill Estimator

Problem statement: Calculate a sample electricity bill using progressive unit slabs.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Units are non-negative. Rates are illustrative and are not an official tariff.

Algorithm: Charge first 100 units at ■1.5, next 100 at ■2.5, and remaining units at ■4.

Text flowchart: Start → Read units → Apply slabs → Calculate total → Print bill → End.

Pseudocode: If units ≤100 calculate first slab; else if ≤200 add second slab; otherwise add all three slabs.

Python source code:

```
units = float(input("Enter electricity units used: "))
if units < 0:
    print("Units cannot be negative")
elif units <= 100:
    bill = units * 1.5
    print("Estimated bill = Rs.", round(bill, 2))
elif units <= 200:
    bill = 100 * 1.5 + (units - 100) * 2.5
    print("Estimated bill = Rs.", round(bill, 2))
else:
    bill = 100 * 1.5 + 100 * 2.5 + (units - 200) * 4
    print("Estimated bill = Rs.", round(bill, 2))
```

Test cases / expected output:

Input	Expected result
80	Estimated bill = Rs. 120.0
150	Estimated bill = Rs. 275.0
250	Estimated bill = Rs. 650.0

Conclusion: The program demonstrates slab-based calculations. The rates are examples for learning only.

Case Study 10: ATM Withdrawal Simulator

Problem statement: Simulate a simple withdrawal from a starting balance.

Objective: Develop a Python solution and test it with representative inputs.

Assumptions: Starting balance and withdrawal amount are non-negative; no bank connection is made.

Algorithm: Check amount, check balance, then subtract the withdrawal if valid.

Text flowchart: Start → Read balance and amount → Validate → Check funds → Update balance → Print → End.

Pseudocode: Read balance and amount; reject invalid values; if amount exceeds balance show error; otherwise show remaining balance.

Python source code:

```
balance = float(input("Enter starting balance: "))
amount = float(input("Enter withdrawal amount: "))
if balance < 0 or amount <= 0:
    print("Enter a non-negative balance and a positive withdrawal amount")
elif amount > balance:
    print("Insufficient balance")
else:
    balance -= amount
    print("Withdrawal successful")
    print("Remaining balance = Rs.", round(balance, 2))
```

Test cases / expected output:

Input	Expected result
Balance 1000; amount 250	Withdrawal successful; Remaining balance = Rs. 750.0

Input	Expected result
Balance 100; amount 150	Insufficient balance

Conclusion: The program applies input validation and conditional logic to a safe, local simulation. It does not connect to a real bank.

Final Checklist Before Submission

■ Compare these provisional case studies with the official list of 100 and replace any topic not listed or approved.

■ Fill in roll number, branch and section.

■ Run every .py program yourself and fix any errors.

■ Capture actual output screenshots and add them to the report or repository.

■ Create a GitHub repository named Python-Activity-Based-Learning.

■ Upload the 10 programs, documentation, screenshots and consolidated PDF.

■ Be ready to explain the logic and demonstrate the programs to your faculty.

Repository README information should include your name, roll number, branch, section and titles of all ten selected case studies, as required by the circular.